



Introducción a JWT y Claims



Contenido

01

Que es JWT

02

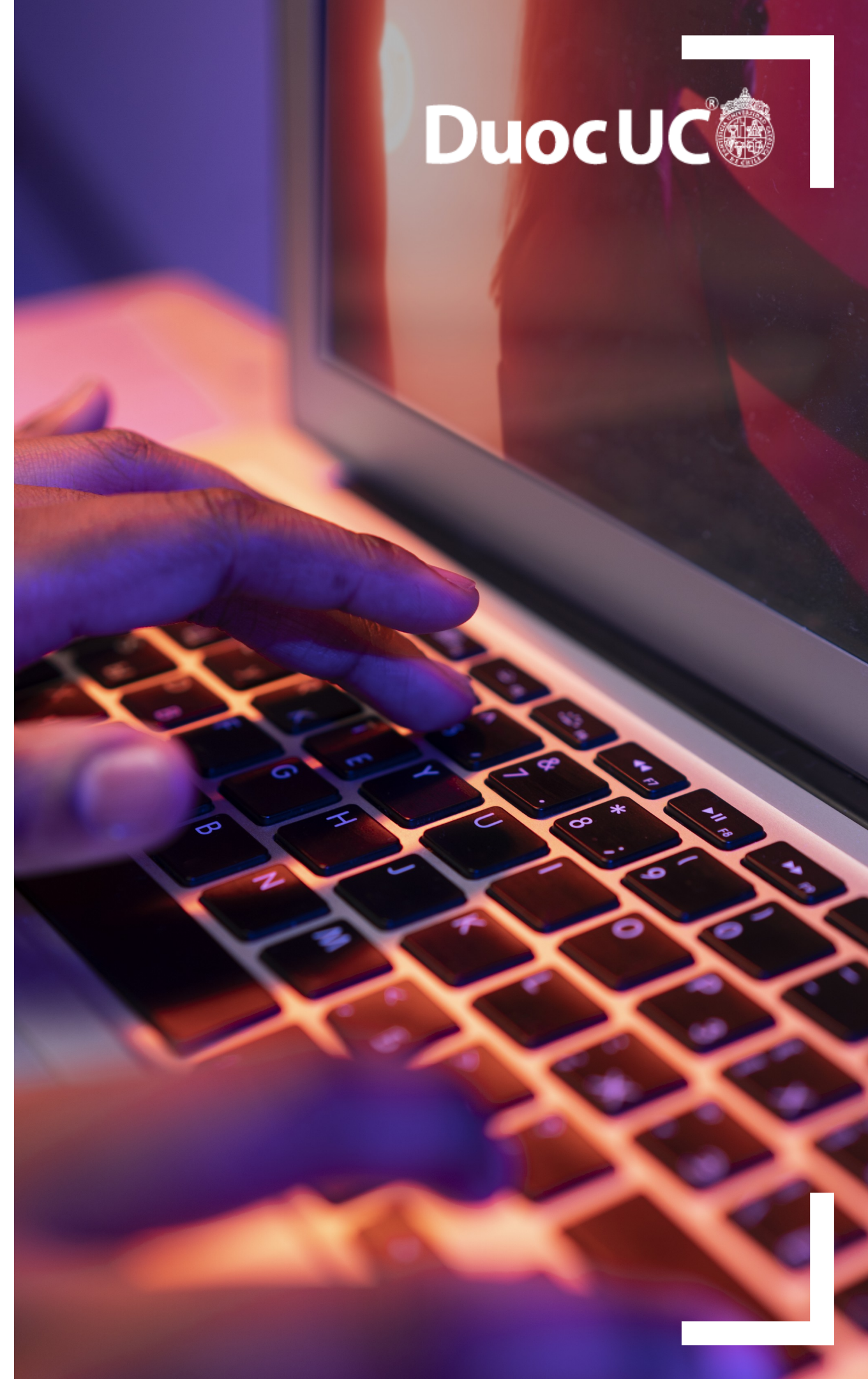
Estructura de JWT

03

Claims

04

Buenas Practica



02

Introducción a JWT y Claims

SUBTÍTULO

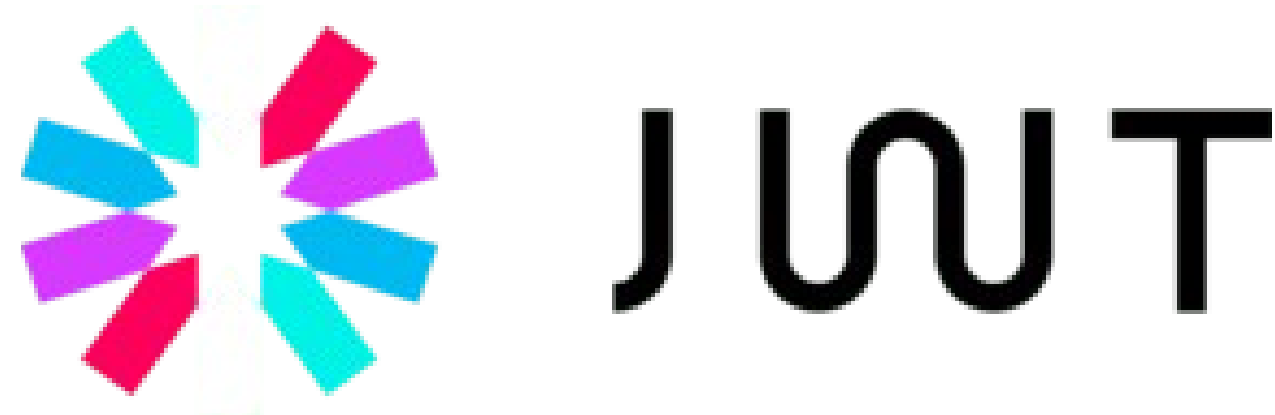
DuocUC 

```
.App-logo {  
  height: 40vmin;  
  pointer-events: none;  
}  
  
@media (prefers-reduced-motion: no-preference) {  
  .App-logo {  
    animation: App-logo-spin infinite 20s linear;  
  }  
}  
  
.App-header {  
  background-color: #282c34;  
  min-height: 100vh;  
  display: flex;  
  flex-direction: column;  
  align-items: center;  
  justify-content: center;  
  font-size: calc(10px + 2vmin);  
  color: white;  
}  
  
.App-link {  
  color: #61dafb;  
}  
  
@keyframes App-logo-spin {  
  from {  
    transform: rotate(0deg);  
  }  
  to {  
    transform: rotate(360deg);  
  }  
}
```


¿Qué es un JWT?

JWT (JSON Web Token) es un token digital firmado que contiene información sobre un usuario o una aplicación.

-  Es como un carnet digital compacto, fácil de transportar entre servicios
-  Autentica usuarios
-  Autoriza acceso a APIs
-  Envía información confiable entre microservicios



¿Por qué existe JWT?

✗ Antes (Sistemas Antiguos)

- Sesión en el servidor
- Guardar estados complejos
- Cookies pesadas
- Difícil en microservicios

✓ JWT (Moderno)

- Stateless
- Viaja con cada request
- Sin BD en servidor
- Perfecto para cloud-native

Lo que hace y lo que NO hace un JWT

✓ Lo que SÍ hace un JWT

- **Garantiza integridad del contenido** gracias a su **firma digital** (HMAC / RSA / ECDSA).
Permite verificar que *nadie alteró* el Header ni el Payload.
- **Transporta información confiable** entre servicios (claims como sub, iss, exp, roles).
- **Permite autenticación y autorización** en APIs, microservicios y aplicaciones web.
- **Es compacto y stateless**, ideal para arquitecturas cloud-native.
- **Evita sesiones server-side**, ya que el estado viaja dentro del token.

Lo que hace y lo que NO hace un JWT

✗ Lo que NO hace un JWT

- **NO cifra la información del payload.**
→ El contenido está solamente **codificado con Base64URL**, que cualquier persona puede decodificar.
- **NO sirve para ocultar datos sensibles.**
→ Nunca guardes en un JWT: contraseñas, RUT, tarjetas, secretos, tokens internos, etc.
- **NO es un mecanismo de confidencialidad.**
→ La seguridad del JWT **no consiste en esconder datos**, sino en asegurar que no se modifiquen.
- **NO garantiza por sí mismo quién está usando el token.**
→ Solo asegura que el token es válido; el uso indebido es posible si se expone.
- **NO reemplaza cifrado.**
→ Si necesitas confidencialidad debes usar:
 - ✓ **HTTPS** (capa de transporte)
 - ✓ **JWE (JSON Web Encryption)** si deseas que el token esté realmente cifrado

El estándar que da vida a JWT

Los JWT forman parte del ecosistema **JOSE (JSON Object Signing and Encryption)**, un conjunto de estándares que define cómo firmar, cifrar y manejar claves en objetos JSON.

Componentes de JOSE:



JWS (JSON Web Signature)

- Define cómo **firmar** un JWT.
- Es lo que realmente usamos cuando hablamos de “JWT firmado”.



JWE (JSON Web Encryption)

- Define cómo **cifrar** un token.
- No es usado por defecto.
- Solo necesario si requieres **confidencialidad real**.



JWK (JSON Web Key)

- Formato estándar para representar una clave pública o privada en JSON.
- Usado por los servidores de identidad para exponer claves mediante el endpoint `/.well-known/jwks.json`.



JWA (JSON Web Algorithms)

- Define los algoritmos válidos para firma y cifrado:
Ej: HS256, RS256, ES256 (firma), RSA-OAEP, A128GCM (cifrado).

La Estructura de JWT



Header

Indica el algoritmo y tipo



Payload

Contiene la información
(claims)



Signature

Garantiza no alteración

La Estructura de JWT

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6ImNsbnV4Iiwicm9sZSI6InN0dWRlbnQpLCJpYXQiOiE3MzIzMTUyMDAsImV4cCI6MTczMjMxODgwMCwiaXNzIjoiaHR0cHM6Ly9hdXRoLmVqZW1wbG8uY2wiLCJhdWQiOiJodHRwczovL2FwaS5lamVtcGxvLmNsIn0.4JjE_t1LTiY9U05weiUEM5Zv5xeQfex5YLSgD7yYtmA

<> HEADER

Algoritmo

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

📄 PAYLOAD

Datos

```
{
  "sub": "1234567890",
  "name": "cloud native 1",
  "role": "student",
  "iat": 1732315200,
  "exp": 1732318800,
  "iss": "https://auth.ejemplo.cl",
  "aud": "https://api.ejemplo.cl"
}
```

** Datos decodificados del token real*

🔒 SIGNATURE

Seguridad

```
HMACSHA256(
  base64Url(header) + "." +
  base64Url(payload),
  🔑 secret
)
```


DECODED HEADER

JSON CLAIMS TABLE

COPY

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

DECODED PAYLOAD

JSON CLAIMS TABLE

HIDE DETAILS

sub	1234567890	The subject of the JWT (the user). Learn more
name	cloud native 1	
role	student	
iat	1732315200 (Fri Nov 22 2024 19:40:00 GMT-0300 (hora de verano de Chile))	The time at which the JWT was issued. Learn more

This value must be a [NumericDate](#) type, representing seconds.

exp	1732318800 (Fri Nov 22 2024 20:40:00 GMT-0300 (hora de verano de Chile))	The expiration time on or after which the JWT MUST NOT be accepted for processing. Learn more
-----	--	---

This value must be a [NumericDate](#) type, representing seconds.

iss	https://auth.ejemplo.cl	The issuer of the JWT. Learn more
aud	https://api.ejemplo.cl	The recipients that the JWT is intended for. Learn more

JWT SIGNATURE VERIFICATION (OPTIONAL)

Enter the secret used to sign the JWT below:

¿Qué son los Claims?

Los claims son "afirmaciones" sobre el usuario o la aplicación. Funciona como un conjunto de datos confiables:

- 👤 Quién eres: sub, name, email
- 🔑 Qué puedes hacer: roles, groups
- 🕒 Cuándo expira: exp
- 📄 Quién emitió: iss
- 📡 Para quién va: aud

[Token de Ejemplo](#)

¿Qué pasa si alguien altera un JWT?

Si un atacante modifica el payload:

- La firma deja de ser válida
- La API lo detecta
- El token es rechazado

Por eso es FUNDAMENTAL:

- Validar la firma SIEMPRE
- Usar JWKS del proveedor
- Nunca confiar sin validar

Enter the secret used to sign the JWT below:

SECRET

COPY CLEAR

signature verification failed

a-string-secret-at-least-256-bits-long

Encoding Format UTF-8 ▼



Buenas Practicas

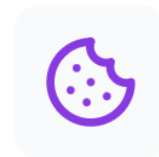
Estrategias clave para asegurar la implementación de JSON Web Tokens en producción.



01

Validar con JWKS siempre

Verifica la firma del token contra las claves públicas del proveedor (Well-Known).



02

Cookies HttpOnly o Memoria

Evita LocalStorage para prevenir ataques XSS (Cross-Site Scripting).



03

Access Tokens Cortos

Reduce la ventana de ataque si un token es robado (ej. 5-15 minutos).



04

Refresh Tokens en Backend

Mantenlos seguros y rota su uso. Nunca exponerlos al cliente si es posible.



05

Encriptar si es Sensible

Usa JWE (JSON Web Encryption) si el payload contiene datos privados (PII).



06

Revocar Tokens Posibles

Implementa listas de revocación o verifica estado en cambios críticos.

Felicitaciones
¡Has finalizado el módulo!

Links de interés

RFC 7519 – JSON Web Token (JWT)

<https://www.rfc-editor.org/rfc/rfc7519>

RFC 7515 – JSON Web Signature (JWS)

<https://www.rfc-editor.org/rfc/rfc7515>

RFC 7516 – JSON Web Encryption (JWE)

<https://www.rfc-editor.org/rfc/rfc7516>

RFC 7517 – JSON Web Key (JWK)

<https://www.rfc-editor.org/rfc/rfc7517>

RFC 7518 – JSON Web Algorithms (JWA)

<https://www.rfc-editor.org/rfc/rfc7518>

IANA – JSON Web Signature and Encryption Algorithms Registry

<https://www.iana.org/assignments/jose/jose.xhtml>

OpenID Connect Core – ID Token Claims

https://openid.net/specs/openid-connect-core-1_0.html#IDToken

DuocUC[®]



CERCANÍA. LIDERAZGO. FUTURO.

7
AÑOS
ACREDITADO

 Comisión Nacional
de Acreditación
CNA-Chile

NIVEL DE EXCELENCIA
HASTA OCTUBRE 2031

Docencia de pregrado / Gestión institucional / Aseguramiento interno de la
calidad / Vinculación con el Medio / Investigación, creación y/o innovación